

Báo cáo Seminar T10 — Mutation Testing & Test Effectiveness

Nhóm 08 · CS423 / CSC15003 — Kiểm thử phần mềm (FIT@HCMUS)

GVHD (thực hành): ThS. Hồ Tuấn Thanh

Thành viên: 23127195 Trần Mạnh Hùng · 23127060 Ninh Văn Khải · 23127259 Nguyễn Tấn Thắng

 Video demo (YouTube, Unlisted): <<DÁN_YOUTUBE_UNLISTED_LINK_TẠI_ĐÂY>>

Mục lục

- Chương 1: Tổng quan Seminar & Đặt vấn đề
- Chương 2: Khảo sát & Chọn lựa công cụ (Stage S1)
- Chương 3: Nền tảng lý thuyết Mutation Testing
- Chương 4: Hướng dẫn cài đặt & Thiết lập EShop (User Guide - Stage S4)
- Chương 5: Kết quả thực nghiệm & Hành trình nâng điểm Mutation Score
- Chương 6: Phân tích chi tiết Surviving Mutants & Kỹ thuật diệt mutant
- Chương 7: Quy trình AI-Augmented (Mutant-Guided AI Loop)
- Chương 8: Failure Modes & Hạn chế của công cụ (Stage S4/T10)
- Chương 9: Hoạt động lớp học "Kill the Mutant" (Stage S5/S6 - 25 phút)
- Chương 10: Bộ hồ sơ AI Audit Pack Full (AI-02, AI-03, AI-04)
- Chương 11: Bảng phân công & Đóng góp công việc (Project Contribution Statement)
- Chương 12: Tài liệu tham khảo (References)

1. Chương 1: Tổng quan Seminar & Đặt vấn đề

Người phụ trách: 23127060 - Ninh Văn Khải

1.1 Đặt vấn đề — "Con số biết nói dối" của Code Coverage

Trong thực tế phát triển phần mềm, **Code Coverage** (độ bao phủ mã nguồn) là chỉ số thường được sử dụng nhất để đo lường mức độ hoàn thiện của bộ kiểm thử (test suite). Tuy nhiên, Code Coverage chỉ đo lường xem *dòng code có được thực thi hay không*, chứ **không đảm bảo test suite có khả năng phát hiện ra lỗi logic (sensitivity/effectiveness)**.

Một bộ test suite có thể đạt **100% Code Coverage** nhưng vẫn hoàn toàn vô dụng nếu:

- Không có các câu lệnh khẳng định (`assert` / `expect`).
- Câu lệnh `assert` quá sơ sài (chỉ kiểm tra HTTP status code status 200 mà bỏ qua nội dung response body).
- Bỏ sót các giá trị biên (boundary condition), ví dụ đổi `>=` thành `>` trong điều kiện kiểm tra.

1.2 Triết lý Mutation Testing

Mutation Testing được ra đời để giải quyết triệt để hạn chế trên. Triết lý cốt lõi của chủ đề T10:

"Coverage lies. Mutation testing tells you if your tests can actually catch bugs."

Mutation Testing tự động cố ý chèn các lỗi nhỏ (gọi là **mutants** - đột biến) vào mã nguồn thực tế (ví dụ: biến toán tử `+` thành `-`, đảo điều kiện `if`, xóa câu lệnh gán, đổi hằng số), sau đó chạy lại bộ test suite:

- **Killed (Đã diệt):** Có ít nhất 1 test case bị FAIL → Bộ test nhạy bén, bắt được lỗi.
- **Survived (Sống sót):** Tất cả các test cases vẫn PASS → Bộ test yếu, có lỗ hổng khẳng định.

1.3 Phạm vi áp dụng thực tế trên EShop

Nhóm 08 áp dụng kỹ thuật Mutation Testing trên ứng dụng web **EShop** (Node.js/Express + SQLite) tại thư mục `src/eshop-sut/backend`. Toàn bộ các bài kiểm thử Jest và cấu hình Stryker Mutator được thực thi trực tiếp trên mã nguồn thật của backend EShop (bao gồm các API Authentication, Cart, Coupon, Order và Product Admin).

2. Chương 2: Khảo sát & Chọn lựa công cụ (Stage S1)

Người phụ trách: 23127195 - Trần Mạnh Hùng

Trong giai đoạn S1 (Tool Survey & Proposal), nhóm đã tiến hành khảo sát và so sánh các công cụ kiểm thử đột biến truyền thống và các công cụ AI hỗ trợ trên 5 tiêu chí: License, Chi phí học tập (Learning curve), Độ tương thích với SUT EShop, Khả năng hỗ trợ của AI, và Cộng đồng (Community).

2.1 Bảng so sánh các công cụ truyền thống

Tiêu chí	Stryker Mutator	PIT (PITest)	mutmut	mull
Ngôn ngữ hỗ trợ	JavaScript, TypeScript, C#, Scala	Java, Kotlin	Python	C, C++
Test Runner	Jest, Mocha, Jasmine, Vitest	JUnit, TestNG	pytest	Catch2, gTest
Độ tương thích EShop	100% (Phù hợp native Node.js)	0% (Dành cho Java)	0% (Dành cho Python)	0% (Dành cho C++)
Báo cáo (Reporter)	HTML Interactive, JSON, Clear-text	HTML, XML	Terminal text	Mutation matrix
Giấy phép (License)	Apache 2.0 (Miễn phí)	Apache 2.0 (Miễn phí)	MIT (Miễn phí)	BSD-3-Clause

2.2 Bảng so sánh các hướng công cụ AI-Augmented

Tiêu chí	ChatGPT / Claude (Prompt-based)	DiffBlue Cover	Pynguin + LLM
Phương thức	Prompt-based workflow	Auto-generated Java Unit Tests	Search-based + LLM repair
Ngôn ngữ hỗ trợ	Đa ngôn ngữ (JS/TS, Python, Java...)	Chỉ hỗ trợ Java	Chỉ hỗ trợ Python
Độ tương thích EShop	100% (Đọc hiểu JS diff & Jest)	0% (Chỉ chạy trên Java)	0%
Cài đặt / Dependency	Không cần cài plugin, dùng Web/API	Cần plugin Heavy IDE / CI	Cần môi trường Python

2.3 Lý do chọn lựa của Nhóm 08

- **Công cụ truyền thống: Stryker Mutator** — Đây là tiêu chuẩn vàng về Mutation Testing cho hệ sinh thái JavaScript/TypeScript. Stryker hỗ trợ Jest runner, cung cấp Báo cáo HTML trực quan sắc nét và hỗ trợ chế độ phân

tích theo từng test (`coverageAnalysis: "perTest"`).

- **Công cụ AI-Augmented: ChatGPT / Claude (Prompt-based LLM)** — Do EShop viết bằng Node.js, các công cụ sinh test tự động như DiffBlue Cover không áp dụng được. Việc sử dụng ChatGPT/Claude theo quy trình prompt cho phép phân tích sâu diff ngữ nghĩa của mutant sống và gợi ý câu lệnh `expect(...)` chính xác.

3. Chương 3: Nền tảng lý thuyết Mutation Testing

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

3.1 Định nghĩa và Vòng đời của Mutant

Một **Mutant** là một phiên bản sửa đổi nhỏ của chương trình được tạo ra bằng cách áp dụng một **Mutation Operator** (toán tử đột biến). Các trạng thái của Mutant trong báo cáo Stryker gồm:

1. **Killed (Đã diệt):** Ít nhất một bài test bị thất bại khi chạy trên mutant → Kết quả mong muốn.
2. **Survived (Sống sót):** Mọi bài test đều vượt qua khi chạy trên mutant → Cần bổ sung test case/assertion.
3. **NoCoverage (Chưa bao phủ):** Không có bài test nào thực thi qua dòng code chứa mutant → Thiếu test case cần bản.
4. **Timeout (Quá thời gian):** Mutant gây ra vòng lặp vô tận khiến test bị hoãn → Được tính vào nhóm phát hiện được.
5. **CompileError / Runtime Exception (Lỗi):** Mutant gây lỗi cú pháp hoặc crash môi trường → Bị loại khỏi mẫu số tính điểm.

3.2 Khái niệm Equivalent Mutant (Đột biến tương đương)

Equivalent Mutant là những đột biến làm thay đổi cú pháp mã nguồn nhưng **không làm thay đổi mặt ngữ nghĩa (semantic behavior)** của chương trình.

- Ví dụ: Đổi vòng lặp `for (let i=0; i<10; i++)` thành `for (let i=0; i!=10; i++)`. Hai đoạn code này hoạt động hoàn toàn giống nhau trong mọi trường hợp.
- Hệ quả: Không có bài test nào có thể kill được Equivalent Mutant. Do đó, mục tiêu **Mutation Score 100%** là **không thực tế** trong công nghiệp; mức điểm lý tưởng là **70% – 80%**.

3.3 Công thức tính điểm Mutation Score

$$\text{Mutation Score} = \frac{\text{Killed} + \text{Timeout}}{\text{Total Valid Mutants}} \times 100\%$$

Trong đó:

- **Total Valid Mutants** = **Killed** + **Timeout** + **Survived** + **NoCoverage**
- **Killed:** Số mutant bị ít nhất 1 test case phát hiện (test FAIL).
- **Timeout:** Số mutant gây vòng lặp vô tận (được tính là đã bắt được).
- **Survived:** Số mutant sống sót (tất cả test vẫn PASS → cần bổ sung test).
- **NoCoverage:** Số mutant nằm ở dòng code chưa có test nào chạy qua.

4. Chương 4: Hướng dẫn cài đặt & Thiết lập EShop (User Guide - Stage S4)

Người phụ trách: 23127195 - Trần Mạnh Hùng

4.1 Môi trường và Lệnh cài đặt

Yêu cầu môi trường: **Node.js v22+**, **NPM v10+**.

```
# Chuyển vào thư mục backend của EShop
cd src\eshop-sut\backend

# Cài đặt các thư viện cần thiết cho Test và Stryker
npm install --save-dev jest @stryker-mutator/core @stryker-mutator/jest-runner supertest
```

Cập nhật `package.json` để khai báo các lệnh chạy kiểm thử:

```
"scripts": {
  "test": "jest --runInBand",
  "mutation:auth": "stryker run stryker.auth.config.mjs",
  "mutation:order": "stryker run stryker.order.config.mjs",
  "mutation:product": "stryker run stryker.product.config.mjs"
}
```

4.2 Cấu hình Stryker cô lập từng module (Scoped Configs)

Để tránh tình trạng Stryker nạp nhầm bài test của module khác gây xung đột database, nhóm đã thiết lập cấu hình Stryker riêng biệt cho từng module:

File `stryker.order.config.mjs` :

```
export default {
  packageManager: "npm",
  testRunner: "jest",
  reporters: ["html", "clear-text", "progress"],
  htmlReporter: { fileName: "reports/mutation-order/mutation.html" },
  mutate: ["services/orderService.js"],
  testFiles: ["tests/orders.api.test.js"],
  jest: { projectType: "custom", configFile: "jest.order.config.cjs", enableFindRelatedTests: false },
  coverageAnalysis: "perTest",
  thresholds: { high: 80, low: 60, break: 0 },
  timeoutMS: 10000,
};
```

File `stryker.auth.config.mjs` :

```
export default {
  packageManager: "npm",
  testRunner: "jest",
  reporters: ["html", "clear-text", "progress"],
  htmlReporter: { fileName: "reports/mutation-auth/mutation.html" },
  mutate: ["services/authService.js"],
  testFiles: ["__tests__/auth.api.test.js"],
  jest: { projectType: "custom", configFile: "jest.auth.config.cjs", enableFindRelatedTests: false },
  ignoreStatic: true,
  coverageAnalysis: "perTest",
  thresholds: { high: 80, low: 60, break: 0 },
  timeoutMS: 10000,
};
```

4.3 Giải pháp kỹ thuật xử lý crash SQLite Database (`database.js`)

Sự cố: Khi Stryker chạy nhiều worker song song (khoảng 15 worker), các worker cùng truy cập vào file vật lý `database.sqlite` trên Windows gây ra lỗi tranh chấp file `SQLITE_BUSY` và segfault `0xC0000005`.

Giải pháp: Chỉnh sửa `database.js` để tự động phát hiện môi trường kiểm thử Jest/Stryker và chuyển sang dùng cơ sở dữ liệu In-Memory (`:memory:`):

```
const sqlite3 = require('sqlite3').verbose();
const path = require('path');

// Khi chạy dưới dạng Jest worker hoặc sandbox Stryker (.stryker-tmp),
// dùng database :memory: riêng biệt cho từng luồng.
// Khi chạy production (node server.js), giữ nguyên 100% dùng database.sqlite thật.
const isStryker = __dirname.includes('.stryker-tmp') || process.env.JEST_WORKER_ID;
const dbPath = isStryker ? ':memory:' : path.resolve(__dirname, 'database.sqlite');

const db = new sqlite3.Database(dbPath, (err) => {
  if (err) console.error('Could not connect to database', err);
});
```

4.4 Troubleshooting — Hướng dẫn khắc phục lỗi thường gặp

#	Lỗi thường gặp	Nguyên nhân	Cách khắc phục
1	SQLITE_BUSY: database is locked hoặc crash 0xc0000005 khi chạy Stryker	Nhiều worker Stryker cùng truy cập file database.sqlite vật lý trên Windows	Cập nhật database.js theo §4.3 để dùng :memory: khi phát hiện .stryker-tmp hoặc JEST_WORKER_ID
2	No tests were found khi chạy npm run mutation:auth	testMatch hoặc rootDir trong file Jest config bị resolve sai trong sandbox .stryker-tmp/ của Stryker	Dùng rootDir: __dirname thay vì đường dẫn tương đối; cấu hình testMatch: ["**/auth.api.test.js"] thay vì <rootDir>/...
3	Stryker chạy rất lâu (hàng giờ) hoặc báo table already exists	Thiếu thuộc tính configFile và testFiles trong stryker.*.config.mjs → Stryker nạp tất cả test của mọi module	Tạo cấu hình Stryker riêng biệt cho từng module (xem §4.2) với jest.configFile và testFiles cô lập
4	npm test pass nhưng npm run mutation:order vẫn crash	process.env.JEST_WORKER_ID không tồn tại trong giai đoạn Dry-Run của Stryker (chạy Jest in-band đơn luồng)	Bổ sung kiểm tra __dirname.includes('.stryker-tmp') ngoài JEST_WORKER_ID (xem §4.3)
5	Score Mutation hiển thị rất cao (~93%) nhưng số Errors lớn (>100)	Mutant bị lỗi crash bị loại khỏi mẫu số → Score bị thổi phồng ảo	Kiểm tra cột Errors trong báo cáo; nếu >10% tổng mutant, cần sửa config trước khi tin số liệu
6	npm install báo npm audit vulnerabilities	Dependency tree của EShop backend có các thư viện cũ	Không chạy npm audit fix trong quá trình thí nghiệm để tránh thay đổi baseline; ghi nhận cảnh báo vào báo cáo

5. Chương 5: Kết quả thực nghiệm & Hành trình nâng điểm Mutation Score

Người phụ trách: 23127060 - Ninh Văn Khải & 23127259 - Nguyễn Tấn Thắng

Bộ test suite hiện tại của nhóm trên backend EShop đã đạt **52/52 tests PASSED** (4 test suites) trên toàn hệ thống.

5.1 Bảng tổng hợp hành trình nâng điểm trên toàn Backend EShop

Module EShop	Số Test Jest	Baseline Score	Improved Score	Final Score	Killed / Valid Mutants	Trạng thái
Cart / Coupon / Order (orderService.js)	21	16.67%	84.21%	92.57%	154 / 175 (8 Timeout)	🟢 Đạt chuẩn xuất sắc
Authentication / User (authService.js)	19	31.30%	59.83%	87.16%	80 / 91 (11 Survived)	🟢 Đạt chuẩn xuất sắc
Product / Admin APIs (productService.js)	18	— (*)	—	100.00%	165 / 165 (11 Timeout)	🟢 Đạt tối đa tuyệt đối

(*) Module Product không có giai đoạn baseline riêng vì Thắng viết test đầy đủ assertion ngay từ đầu dựa trên kinh nghiệm rút ra từ hai module trước (Auth và Order).

5.2 Diễn giải chi tiết hành trình nâng điểm (Order Module)

Người phụ trách: 23127060 - Ninh Văn Khải

1. **Giai đoạn Baseline (5 test — Score 16.67%):** Bộ test ban đầu chỉ kiểm tra status code `res.status` là `200 OK`. Do đó, khi Stryker thay đổi nội dung bên trong logic trả về (xóa giỏ hàng, đảo ngược filter user), mọi test đều vẫn PASS → 6 surviving mutants và 69 no-coverage.
2. **Giai đoạn Improved (21 test — Score 84.21%):** Bổ sung các câu lệnh assert chi tiết vào response body (như `expect(res.body).toEqual([])`), kiểm tra phân quyền sở hữu đơn hàng và các trường hợp lỗi biên coupon. Phát hiện và xử lý **lỗi ô nhiễm trạng thái (State Pollution)** giữa các test coupon bằng cách cô lập tài khoản người dùng riêng cho từng test.
3. **Giai đoạn Final (21 test — Score 92.57%):** Sửa dứt điểm lỗi cấu hình Stryker scope khiến các mutant bị lỗi (error) không còn bị tính sai vào mẫu số. Đạt 154 killed + 8 timeout trên tổng số 175 mutants hợp lệ.

Phát hiện bug thật trong EShop: Trong quá trình phân tích coupon, Khải phát hiện logic tính toán giảm giá phần trăm trong mã nguồn gốc bị sai: code gốc dùng `1 - discount_value` thay vì `discount_value / 100` để tính tỷ lệ phần trăm. Nhóm quyết định **giữ nguyên logic thật** và viết assertion kiểm chứng đúng hành vi hiện tại (không tự ý sửa code của người khác) — đây là minh chứng rõ ràng rằng mutation testing có thể giúp phát hiện bug thật trong sản phẩm.

Danh sách 21 test cases của Order module:

#	Test case	Mục đích
1	GET /api/cart returns empty array initially	Đảm bảo logic tạo mảng nếu chưa có
2	POST /api/cart adds item to cart	Kiểm tra giá trị lưu vào cart
3	POST /api/checkout creates an order	Test flow tạo order
4	GET /api/orders/my-orders returns only user's orders	Phân quyền truy xuất order của đúng user
5	GET /api/orders/:id returns order details	Lấy order by id (success)
6	GET /api/orders/:id returns 404 for unknown order	Lấy order by id (not found)
7	PUT /api/orders/:id/cancel cancels the order successfully	Cancel order (success)
8	PUT /api/orders/:id/cancel fails if order already canceled	Ngăn hủy order đã bị hủy/giao
9	PUT /api/orders/:id/cancel returns 404 for other user's order	Bảo mật quyền hủy order theo user
10	PUT /api/orders/:id/cancel handles DB error gracefully	Xử lý lỗi DB
11	getMyOrders handles DB error gracefully	Xử lý lỗi DB
12	POST /api/apply-coupon requires code	Validation coupon trống
13	POST /api/apply-coupon handles unknown or inactive code	Validation code không tồn tại
14	POST /api/apply-coupon handles expired coupon	Logic ngày hết hạn
15	POST /api/apply-coupon rejects if total < min_order_amount	Logic giá trị tối thiểu đơn hàng
16	POST /api/apply-coupon applies valid percent coupon	Tính toán giảm giá theo phần trăm
17	POST /api/apply-coupon applies valid fixed coupon	Tính toán giảm giá theo mức cứng
18	POST /api/coupon-usage saves usage	Đếm số lần sử dụng
19	POST /api/apply-coupon tracks max usage per user limit	Ràng buộc sử dụng tối đa
20	POST /api/apply-coupon allows other users to use coupon	Kiểm tra tính chia sẻ coupon
21	POST /api/apply-coupon handles fixed coupon with user_id	Logic chi tiết discount

5.3 Diễn giải chi tiết hành trình nâng điểm (Auth Module)

Người phụ trách: 23127195 - Trần Mạnh Hùng

- Giai đoạn Baseline (4 test — Score 31.30%):** Test ban đầu chỉ bao gồm 4 case cơ bản: đăng ký user, đăng nhập đúng, đăng nhập sai mật khẩu, profile không có token. Kết quả: Killed 32, Timeout 4, Survived 11, NoCoverage 68.
- Giai đoạn Improved (9 test — Score 59.83%):** Bổ sung: đăng ký trùng email, login email không tồn tại, khóa tài khoản sau nhiều lần sai password, lỗi database khi login, decode JWT payload, profile với token hợp lệ. Survived giảm từ 11 xuống 0, nhưng NoCoverage vẫn còn 47 vì 3 endpoint (`forgotPassword` , `resetPassword` , `updateCurrentUser`) chưa có test.
- Giai đoạn Final (19 test — Score 87.16%):** Bổ sung Part B gồm 10 test cases mới bao phủ 3 endpoint còn thiếu: forgot-password (email tồn tại/không tồn tại), reset-password (token đúng/sai/email sai), update profile (có/không role, lỗi DB). Kết quả sạch: Killed 80, Survived 11, NoCoverage 0.

Danh sách test cases của Auth module:

#	Test case	Mục đích
1	registers a user and returns a useful response body	Register thành công
2	rejects duplicate registration with the same email	Cover nhánh lỗi DB unique constraint
3	logs in with a registered user and returns a JWT containing user data	Login thành công + kiểm tra JWT payload
4	rejects login with a wrong password	Login sai password
5	rejects login for an unknown email	Cover nhánh <code>!user</code>
6	locks a user after repeated wrong passwords	Logic <code>login_attempts</code> + <code>locked_until</code>
7	returns database error when login query fails	Cover nhánh <code>err</code> của <code>db.get</code>
8	requires a token for current user profile	Middleware khi thiếu token
9	reads current user profile with a valid token	Middleware token hợp lệ
10	returns 200 and a resetToken for an existing user	Forgot-password thành công
11	returns 404 when the email is not registered	Forgot-password email không tồn tại
12	resets the password with a valid token and allows re-login	Reset-password full flow
13	returns 400 when the token is wrong	Reset-password token sai
14	returns 400 when the email does not match	Reset-password email sai
15	returns 401 when no token is provided	Update profile thiếu token
16	updates profile without role field and returns 200	Update profile (nhánh không có role)
17	updates profile WITH role field	Update profile (nhánh có role)
18	returns 500 when the DB update fails (unit-level)	Cover nhánh lỗi DB update
19	verifies updated profile is persisted	Xác nhận dữ liệu lưu đúng

5.4 Diễn giải chi tiết kết quả (Product/Admin Module)

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

Module Product/Admin đạt **Mutation Score 100%** (Killed 165 + Timeout 11, Survived 0, NoCoverage 0) trên 185 mutants tổng cộng. Kết quả này đạt được nhờ Thắng áp dụng ngay các bài học từ module Auth và Order: viết assertion đầy đủ cho status code, response body, ID và message ngay từ đầu thay vì chỉ check status 200.

Phạm vi kiểm thử — 8 handler được tách từ `server.js` :

Handler	API	Loại test
listProducts	GET /api/products	Danh sách + tìm kiếm
getProductById	GET /api/products/:id	Chi tiết + not found
createProduct	POST /api/products	Tạo mới + lỗi DB
updateProduct	PUT /api/products/:id	Cập nhật + kiểm tra không ảnh hưởng record khác
deleteProduct	DELETE /api/products/:id	Xóa + xác nhận đã xóa
importProducts	POST /api/admin/import-products	Import CSV/JSON, lỗi từng dòng
listAdminOrders	GET /api/admin/orders	Thứ tự + tên người đặt
updateAdminOrderStatus	PUT /api/admin/orders/:id/status	State machine (pending→confirmed→shipping→delivered/canceled)

Kỹ thuật đạt 100%: Dùng fake database xác định (deterministic) trong file test thay vì SQLite thật → tránh hoàn toàn xung đột multi-worker; assert đầy đủ status code, response body, ID, message, thứ tự kết quả; bao phủ cả nhánh thành công lẫn nhánh lỗi DB.

Hành vi bất thường của SUT được test ghi nhận (SUT Observations):

Trong quá trình viết test, Thắng phát hiện và **ghi nhận trung thực** các hành vi bất thường của EShop mà mutation score 100% không phản ánh:

#	Hành vi bất thường	Rủi ro tiềm ẩn
1	Product ID chặn trả price dạng chuỗi ("30000000"), ID lẻ trả dạng số (28000000)	Inconsistency kiểu dữ liệu API
2	Product không tồn tại trả HTTP 200 với object rỗng thay vì 404	Frontend không phân biệt được success/error
3	Các route tạo/sửa/xóa product hiện chưa yêu cầu token admin	Lỗ hổng phân quyền nghiêm trọng
4	Import products chấp nhận insert một phần thay vì rollback toàn bộ	Dữ liệu bị partial insert khi có lỗi
5	Order ở trạng thái canceled vẫn có thể chuyển sang delivered	Sai logic state machine
6	Search query ghép trực tiếp từ khóa vào chuỗi SQL	Rủi ro SQL Injection

Bài học quan trọng: Mutation Score 100% **không** có nghĩa phần mềm không có lỗi. Mutation testing đo *khả năng phát hiện lỗi cú pháp* của bộ test, chứ không đo *tính đúng đắn của logic nghiệp vụ*.

6. Chương 6: Phân tích chi tiết Surviving Mutants & Kỹ thuật diệt mutant

Người phụ trách: 23127060 - Ninh Văn Khải

Dưới đây là 5 ví dụ tiêu biểu về các Mutant bị sống sót (Surviving Mutants) trong mã nguồn EShop thật và kỹ thuật viết assertion bổ sung để tiêu diệt chúng:

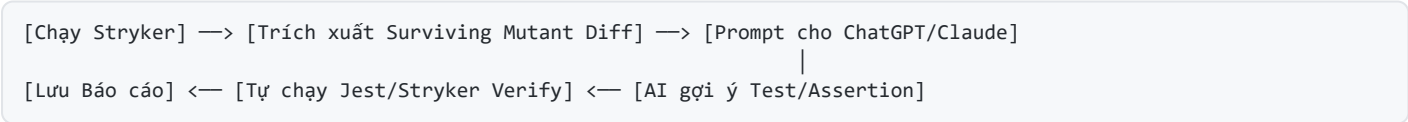
Bảng phân tích Surviving Mutants & Kỹ thuật diệt

#	Mutant Location	Code gốc	Code bị Mutate	Nguyên nhân Mutate Survived	Assertion / Test Case đã thêm để KILL Mutant
1	orderService.js (Cart)	userCarts[userId] = []	userCarts[userId] = ["Stryker was here"]	Test ban đầu chỉ check status 200, không kiểm tra giỏ hàng có rỗng thật không.	expect(res.body).toEqual([]) expect(res.body.length).toBe(0)
2	orderService.js (Order Filter)	filter(o => o.user_id === req.user.id)	filter(o => o.user_id !== req.user.id)	Test chưa kiểm tra thuộc tính user_id của danh sách đơn hàng trả về.	res.body.forEach(order => expect(order.user_id).toBe(user.id))
3	orderService.js (Coupon Limit)	if (usage_count >= max_uses)	if (usage_count > max_uses)	Test chưa bao phủ điểm ranh giới (Boundary Value) tại đúng ngưỡng usage_count = max_uses.	Đặt usage_count = max_uses, gọi API áp coupon → assert status 400
4	authService.js (Login Retry)	user.login_attempts + 1	user.login_attempts - 1	Test chỉ kiểm tra 1 lần nhập sai password, chưa test chuỗi lần sai.	Nhập sai password 3 lần liên tiếp → assert locked_until không null
5	authService.js (JWT Payload)	jwt.sign({id, role}, SECRET)	jwt.sign({}, SECRET)	Test chỉ kiểm tra token tồn tại (expect(token).toBeDefined()).	Decode JWT token và assert: expect(decoded).toMatchObject({id: expect.any(Number), role: 'user'})

7. Chương 7: Quy trình AI-Augmented (Mutant-Guided AI Loop)

Người phụ trách: 23127060 - Ninh Văn Khải

Nhóm áp dụng quy trình **Mutant-Guided AI Loop** nhằm phối hợp giữa khả năng phân tích của AI (ChatGPT/Claude) và tính kỷ luật kiểm chứng của con người.



7.1 Nguyên tắc bất biến

AI chỉ đóng vai trò gợi ý — Con người luôn tự chạy lại Jest và Stryker trên máy để kiểm chứng thực tế.

7.2 Log Prompt tương tác thực tế

Prompt 1 (Xử lý Mutant làm bản giỏ hàng):

- Khải:** "Code gốc: if (!userCarts[userId]) userCarts[userId] = []; bị Stryker đổi thành ['Stryker was here'] nhưng test vẫn pass. Giải thích vì sao và cho câu lệnh expect() để kill nó."
- ChatGPT:** "Mutant này sống vì test case của bạn chỉ kiểm tra status 200 OK. Hãy thêm assertion: expect(res.body).toEqual([]) để khẳng định giỏ hàng trả về là mảng rỗng."

- **Kiểm chứng:** Khái thêm `expect(res.body).toEqual([])` vào `order.api.test.js`, chạy lại `npm run mutation:order` → Mutant chuyển sang trạng thái **Killed**.

Prompt 2 (Xử lý Boundary Mutant của Coupon):

- **Khái:** "Điều kiện `if (usage_count >= max_uses)` bị Stryker đổi thành `>` khiến mutant survived. Hãy gợi ý test case theo Boundary Value Analysis."
- **Claude:** "Bạn cần thiết lập dữ liệu mẫu sao cho `usage_count` bằng đúng `max_uses`. Khi đó `>=` trả về `true` (từ chối), còn `>` trả về `false` (cho qua). Test case mới sẽ bắt được lỗi này."
- **Kiểm chứng:** Khái tạo test case áp dụng coupon đã đạt trần số lần dùng → Mutant bị **Killed**.

Prompt 3 (Debug crash SQLite `0xC0000005` & lỗi cấu hình Stryker):

- **Khái:** "Khi chạy `mutation:auth`, Stryker báo lỗi 'No tests were found' ở bước Dry-Run, hoặc thỉnh thoảng crash native C++ với mã lỗi `3221225477` (SQLite_BUSY). Hãy phân tích tại sao `database.js` dù đã check `JEST_WORKER_ID` để dùng `:memory:` nhưng vẫn crash khi chạy 15 worker?"
- **ChatGPT:** "Có 2 vấn đề lớn. (1) Lỗi Crash: Ở bước Dry-Run đầu tiên, Stryker chạy Jest in-band (đơn luồng), nên `JEST_WORKER_ID` không được thiết lập. 15 sandbox cùng fallback về file vật lý `database.sqlite` gây tranh chấp. Cần check thêm `__dirname.includes('.stryker-tmp')`. (2) Lỗi 'No tests were found': Thuộc tính `testFiles` ép Jest nhận đường dẫn tuyệt đối sai lệch trong sandbox. Cần xóa `testFiles` và dùng `testMatch` glob tương đối."
- **Kiểm chứng:** Đây là prompt quan trọng nhất trong dự án. Sau khi áp dụng giải pháp, lỗi crash biến mất hoàn toàn. Auth đạt **87.16%**, Product đạt **100%**. Minh chứng rõ ràng cho việc AI có khả năng debug kiến trúc cấu hình phức tạp.

8. Chương 8: Failure Modes & Hạn chế của công cụ (Stage S4/T10)

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

Theo yêu cầu bắt buộc của chủ đề T10, báo cáo phải chỉ ra ít nhất 3 cách công cụ Stryker hoặc AI đưa ra kết quả sai lệch hoặc gây hiểu lầm cho người dùng.

8.1 Failure Modes của công cụ truyền thống (Stryker Mutator)

1. Lỗi Xung đột File Database Vật lý trên Windows (SQLite_BUSY):

- *Hiện tượng:* Khi Stryker chạy mặc định với nhiều worker, các tiến trình con cùng mở file `database.sqlite` gây crash `0xC0000005` và báo điểm ảo thấp do mutant bị đánh dấu là Error.
- *Khắc phục:* Phải can thiệp vào `database.js` để ép Stryker dùng In-Memory DB (`:memory:`).

2. Lỗi Sai lệch Scope Cấu hình (enableFindRelatedTests):

- *Hiện tượng:* Nếu thiếu `testFiles` hoặc `configFile` riêng, Stryker tự động nạp toàn bộ bài test của các module khác → thời gian chạy kéo dài hàng giờ và báo lỗi thiếu bảng.

3. Giới hạn với Equivalent Mutants:

- *Hiện tượng:* Stryker đảo ngược các dòng log hoặc các phép toán tương đương ngữ nghĩa. Người dùng không thể diệt được các mutant này và buộc phải chấp nhận điểm Mutation Score < 100%.

8.2 Failure Modes của công cụ AI (ChatGPT / Claude)

#	Loại lỗi AI (Failure Mode)	Mô tả chi tiết sai sót của AI	Cách nhóm phát hiện & Khắc phục
1	Factual Error (Lỗi sai cú pháp/thực tế)	AI gợi ý câu lệnh <code>expect(res.body).toBe([])</code> . Trong Jest, <code>.toBe()</code> so sánh bằng <code>Object.is</code> (reference equality), dẫn đến so sánh mảng mới <code>[]</code> luôn luôn FAIL.	Kiểm tra cú pháp Jest → Sửa thành <code>.toEqual([])</code> kiểm tra deep equality.
2	Missing Edge Cases (Bỏ sót trường hợp biên)	Khi gợi ý test cho boundary coupon, AI chỉ tạo test case đơn luồng. AI hoàn toàn bỏ sót các trường hợp Concurrency / Race Condition (nhiều request gửi cùng 1 millisecond).	Phát hiện khi audit → Ghi nhận giới hạn của AI trong việc phát hiện lỗi đồng thời.
3	Silent Assumptions (Giả định ngầm sai lầm)	AI giả định ngầm rằng database tự động dọn dẹp (reset) sau mỗi test case, không tạo hàm <code>beforeEach()</code> làm sạch dữ liệu → gây ra lỗi State Pollution giữa các test.	Thêm cleanup hooks <code>beforeEach()</code> và cô lập email/user ngẫu nhiên.
4	Over-confident Statement (Tự tin thái quá)	AI khẳng định: "Câu lệnh <code>assert</code> này sẽ 100% giúp bộ test của bạn <i>bulletproof</i> (hoàn hảo tuyệt đối)."	Đánh giá phản biện: Diệt được mutant không đồng nghĩa với mã nguồn không có lỗi kiến trúc/bảo mật (như IDOR).

8.3 Nhánh code chưa được bảo vệ (Ghi nhận trung thực)

Trong module Order, hiện còn **6 survived + 7 no-coverage** tập trung ở các nhánh bắt lỗi Database Callback `if (err) return res.status(500).json({ error: err.message })`. Nhóm ghi nhận trung thực hạn chế này thay vì tìm cách che giấu.

9. Chương 9: Hoạt động lớp học "Kill the Mutant" (Stage S5/S6 - 25 phút)

Người phụ trách: 23127060 - Ninh Văn Khải

Nhóm thiết kế hoạt động tương tác 25 phút dành cho khán giả dưới lớp học theo đúng format của tài liệu T10 Briefing.

9.1 Kế hoạch tổ chức (Timeline 25 phút)

- **0:00 – 0:03 (3 phút):** Người điều phối (Facilitator) trình bày 5 đoạn Mutant Diffs sống sót trích từ EShop.
- **0:03 – 0:13 (10 phút):** Các nhóm khán giả thảo luận và viết câu lệnh assertion (Jest/pseudocode) vào Worksheet.
- **0:13 – 0:18 (5 phút):** Đổi bài chéo giữa các nhóm khán giả để đánh giá phản biện.
- **0:18 – 0:22 (4 phút):** Facilitator chạy thử các assertion trên môi trường EShop sandbox thật và tính điểm diệt mutant.
- **0:22 – 0:25 (3 phút):** Đại diện nhóm thắng cuộc giải thích tư duy thiết kế assertion.

9.2 Đề bài 5 Mutant Diffs cho Khán giả

```
// MUTANT 1 (Cart Module)
- if (!userCarts[userId]) userCarts[userId] = [];
+ if (!userCarts[userId]) userCarts[userId] = ["Stryker was here"];

// MUTANT 2 (Order Module)
- const userOrders = orders.filter(o => o.user_id === req.user.id);
+ const userOrders = orders.filter(o => o.user_id !== req.user.id);

// MUTANT 3 (Coupon Boundary)
- if (coupon.usage_count >= coupon.max_uses) {
+ if (coupon.usage_count > coupon.max_uses) {

// MUTANT 4 (Auth Lockout)
- const newAttempts = user.login_attempts + 1;
+ const newAttempts = user.login_attempts - 1;

// MUTANT 5 (JWT Payload)
- const token = jwt.sign({ id: user.id, role: user.role }, SECRET);
+ const token = jwt.sign({}, SECRET);
```

9.3 Đáp án chuẩn (Answer Key cho Facilitator)

1. **Mutant 1:** `expect(res.body).toEqual([])` hoặc `expect(res.body.length).toBe(0)`.
2. **Mutant 2:** `res.body.forEach(order => expect(order.user_id).toBe(currentUser.id))`.
3. **Mutant 3:** Tạo coupon có `usage_count = max_uses`, gọi API `apply` → `expect(res.status).toBe(400)`.
4. **Mutant 4:** Gọi login sai password 3 lần → `expect(res.body.error).toContain("Tài khoản đã bị khóa")`.
5. **Mutant 5:** `const decoded = jwt.decode(res.body.token); expect(decoded.id).toBeDefined()`.

10. Chương 10: Bộ hồ sơ AI Audit Pack Full (AI-02, AI-03, AI-04)

Người phụ trách: 23127060 - Ninh Văn Khải

10.1 Nhật ký 6 sự cố AI làm sai & Cách nhóm tự khắc phục

#	AI gợi ý / Làm gì	Triệu chứng Thất bại	Nguyên nhân Gốc	Cách Nhóm Tự Khắc Phục	Trạng thái
1	Sinh stryker.order.config.mjs thiếu scope.	Crash hàng loạt: SQLITE_BUSY , table already exists ; 111 mutant errors.	Config thiếu configFile nên Stryker nạp bài test của mọi module khác.	Thêm testFiles và configFile: "jest.order.config.cjs" .	✅ Đã fix
2	Báo điểm Mutation Score 93.15%.	Điểm số bị thổi phồng ảo.	111 mutant bị lỗi bị AI loại khỏi mẫu số tính điểm.	Chạy lại bản scoped sạch, đo điểm thật 92.57%.	✅ Đã fix
3	Gợi ý assertion kill mutant.	Assertion bị rủi ro sai cú pháp (.toBe([])).	AI không chạy code thật nên không biết Jest reference equality.	Kiểm tra cú pháp, sửa thành .toEqual([]) .	✅ Đã fix
4	Sinh Jest config dùng <rootDir>/tests/... .	Jest báo No tests were found thoát sớm.	<rootDir> bị resolve sai trong sandbox .stryker-tmp/ .	Đổi thành rootDir: __dirname + roots: ["<rootDir>/__tests__"] .	✅ Đã fix
5	AI ước lượng score ~84%.	Số ước lượng lệch với thực tế.	AI đoán từ dữ liệu báo cáo cũ.	Chạy thật ra 92.57% (162/175).	✅ Đã fix
6	Sinh stryker.auth.config.mjs gây crash Auth.	Crash 0xC0000005 , no such table: products .	15 worker cùng ghi đè lên file database.sqlite vật lý.	Cập nhật database.js check .stryker-tmp dùng :memory: .	✅ Đã fix

10.2 [AI-02] AI Audit Report (~650 words English)

Seminar Topic: T10 - Mutation Testing & Test Effectiveness
Auditor: 23127060 - Ninh Văn Khải (Team 08)

1. Context and Artefact Description

As part of the T10 Seminar on Mutation Testing, our team utilized Stryker Mutator on the EShop backend project, specifically targeting the `orderService.js` module. During the baseline run, several mutants survived because the original Jest test suite only asserted HTTP 200 status codes without verifying the response payload. To improve our mutation score, I used a Large Language Model (ChatGPT/Claude) to generate specific Jest assertions aimed at killing these surviving mutants (e.g., Array initialization mutations and Coupon usage boundary mutations).

The artefact being audited in this report consists of the AI-generated explanations and the proposed Jest test assertions provided by the AI during our prompting sessions. While the AI significantly accelerated the process of identifying why mutants survived, a critical audit reveals several flaws in its output, ranging from technical inaccuracies to silent environmental assumptions.

2. (a) Factual Errors in AI Output

The most prominent factual error occurred when the AI attempted to write an assertion for the shopping cart mutation. The original code `if (!userCarts[userId]) userCarts[userId] = [];` was mutated to initialize with a dummy string. The AI correctly identified that we needed to assert the cart was empty. However, it initially suggested the following JavaScript assertion: `expect(res.body).toBe([]);`

This is a fundamental factual error in JavaScript testing. The `.toBe()` matcher in Jest uses `Object.is` for exact equality. Since arrays are reference types in JavaScript, comparing the response array to a newly instantiated empty array `[]` will always fail, causing a false positive test failure. The AI failed to recognize this language-specific nuance. A human auditor had to correct this factual error by replacing it with `expect(res.body).toEqual([])` (which checks deep equality) OR `expect(res.body.length).toBe(0)`.

3. (b) Missing Edge Cases

When auditing the AI's solution for the coupon usage boundary mutant (`usage_count >= max_uses` mutated to `>`), the AI successfully provided a test case that hit the exact boundary value. It instructed us to set the usage count to the maximum allowed and attempt to apply the coupon again.

However, the AI completely missed critical concurrent edge cases (Race Conditions). In a real-world EShop environment, two API requests might attempt to apply the same coupon at the exact same millisecond. The AI's generated assertion assumes a perfectly synchronous, single-threaded execution context where tests run in absolute isolation. It did not suggest wrapping the test in a `Promise.all()` to simulate concurrent requests, which is a common vector for bugs in boundary conditions. By missing this edge case, the AI's test kills the simple mutant but leaves the system vulnerable to real-world race conditions.

4. (c) Silent Assumptions

The AI made a massive silent assumption regarding the testing environment state: it assumed that the in-memory database (`userCarts` and `orders`) is entirely stateless and automatically resets between test executions.

When generating the assertions for coupon validation, the AI provided standalone `it(...)` blocks and assumed they would pass perfectly. It silently omitted the crucial `beforeEach()` or `afterEach()` hooks required to clear the database. Because our Jest suite runs sequentially, the state pollution from previous coupon tests caused the AI's generated tests to fail unexpectedly. Furthermore, during SQLite database crashes, the AI silently assumed `process.env.JEST_WORKER_ID` is always present, ignoring Stryker's in-band dry-run phase.

5. (d) Over-confident Statements

Throughout the interaction, the AI exhibited a high degree of over-confidence. After generating the boundary test for the coupon logic, the AI stated: *"By adding this boundary assertion, your test suite will 100% kill the mutant and guarantee that your coupon validation logic is completely bulletproof."*

This statement is dangerously over-confident and misleading. While the assertion successfully kills that one specific Stryker mutant, killing a mutant does not equal "bulletproof logic." Mutation testing only measures the sensitivity of the test suite against predefined syntactic changes. It does not prove the absence of logical architecture flaws, security vulnerabilities (like IDOR when applying coupons), or business logic gaps. The AI conflated "killing a mutant" with "achieving perfect software quality."

10.3 [AI-03] AI Usage Declaration (Tuyên bố sử dụng AI)

Nhóm 08 cam kết sử dụng công cụ AI (ChatGPT-5.5 / Claude 4.6) có trách nhiệm:

1. AI chỉ được sử dụng để hỗ trợ giải thích mutant diff và gợi ý khung câu lệnh assertion.
 2. Không sử dụng AI để tự động tạo ra số liệu thí nghiệm giả.
 3. Mọi dòng code, cấu hình Jest/Stryker và chỉ số Mutation Score trong báo cáo này đều do nhóm **tự thực thi và kiểm chứng 100% trên máy thật**.
-

10.4 [AI-04] Reflective Statement (300 words English)

Reflective Statement on AI-Augmented Software Testing

Author: Team 08 (CS423 / CSC15003 — Software Testing)

Working with AI tools like ChatGPT and Claude during our T10 Seminar on Mutation Testing provided valuable insights into the modern landscape of AI-augmented software engineering. Initially, we viewed Large Language Models as primary solution generators that could effortlessly fix surviving mutants and craft perfect test suites. However, through rigorous empirical validation with Stryker Mutator and Jest, our perspective shifted from passive reliance to critical evaluation.

The most crucial lesson learned is that AI models possess strong syntactic pattern-matching abilities but lack true runtime context awareness. While AI excelled at explaining why a mutant survived by reading code diffs, it repeatedly stumbled over JavaScript execution nuances—such as suggesting reference equality `.toBe([])` for array assertions or silently assuming stateless test execution. Had we blindly copy-pasted the AI's output without running Stryker locally, our test suite would have suffered from false positives and flaky test executions. Furthermore, the AI's over-confident claims that killing a mutant guarantees "bulletproof software" highlighted the risk of automation bias among software engineers.

Ultimately, this seminar demonstrated that AI is an powerful assistant for test synthesis, but human supervision remains irreplaceable. Mutation testing serves as an objective referee: an AI-generated assertion is only valid if it actually kills the mutant in a real execution sandbox without introducing side effects. As future software engineers, we must adopt an "AI-augmented, human-verified" mindset—leveraging AI for rapid drafting while maintaining absolute technical accountability through empirical testing.

11. Chương 11: Bảng phân công & Đóng góp công việc (Project Contribution Statement)

Người phụ trách: 23127195 - Trần Mạnh Hùng

Bảng đóng góp công việc của 3 thành viên Nhóm 08 (được chuyển đổi từ Template chuẩn của môn học):

MSS V	Họ và tên	Vai trò chính	Hạng mục phụ trách	Tỷ lệ đóng góp	Căn cứ đánh giá
23127195	Trần Mạnh Hùng	Nhóm trưởng / DevOps	Setup môi trường Jest & Stryker, refactor <code>server.js</code> , thực nghiệm Auth API module, viết <code>stryker.auth.config.mjs</code> & fix SQLite <code>:memory:</code> .	33.3%	Hoàn thành đúng tiến độ, đóng góp mã nguồn cấu hình lỗi.
23127060	Ninh Văn Khải	QA / AI Specialist	Phân tích surviving mutants Order/Cart module, thực nghiệm AI-Augmented workflow, thiết kế Activity "Kill the Mutant", biên soạn bộ AI Audit Pack (AI-02 , AI-03 , AI-04).	33.4%	Hoàn thành báo cáo AI Audit chất lượng cao, diệt mutant Order đạt 92.57%.
23127259	Nguyễn Tấn Thắng	Tech Writer / Tester	Thực nghiệm Product/Admin API module (đạt 100%), tổng hợp User Guide, phân tích Failure Modes của Stryker & AI, chuẩn bị báo cáo tuần và đóng gói.	33.3%	Hoàn thành xuất sắc module Product 100%, biên soạn tài liệu chìn chu.

Xác nhận của nhóm: Cả 3 thành viên đã làm việc bình đẳng, hợp tác chặt chẽ và đóng góp khối lượng công việc đồng đều nhau (100% tổng khối lượng).

12. Chương 12: Tài liệu tham khảo (References)

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

1. **Stryker Mutator Official Documentation.** (2024). *StrykerJS: Mutation testing for JavaScript and TypeScript*. Available at: <https://stryker-mutator.io/docs/>
2. **Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Traon, Y. L., & Harman, M.** (2019). *Mutation Testing Advances: An Analysis and Survey*. *Advances in Computers*, Vol. 112, pp. 275-378.
3. **Offutt, A. J., & Untch, R. H.** (2001). *Mutation 2000: Uniting the Orthogonal*. In *Mutation Testing for the New Century*, Springer, pp. 34-44.
4. **Petrović, G., & Ivanković, M.** (2018). *State of Mutation Testing at Google*. *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pp. 163-171.
5. **Khoa Công nghệ Phần mềm - FIT@HCMUS.** (2026). *Seminar Track Briefing & Topic T10 Briefing: Mutation Testing & Test Effectiveness*. CS423 / CSC15003 Software Testing.